Welcome to the minitutorial!

To encourage interaction, please come to the ground floor and site near the front row.

https://github.com/PerformanceEstimation/Tutorial-SIAM-OP26

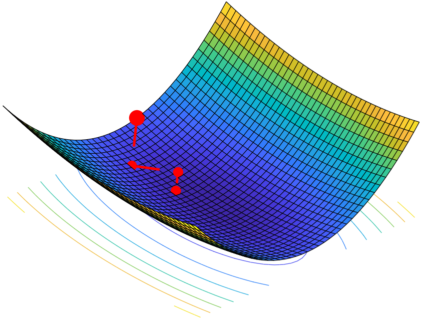
Performance Estimation Problems (PEPs)

Systematic analysis and design of first-order optimization algorithms via the performance estimation framework

François Glineur, Adrien Taylor

SIAM Conference on Optimization, tutorial session – June 2026

$\min_{x \in \mathbb{R}^d} f(x)$ $\rightarrow$ usually solved via **iterative algorithm** generating sequence $x_0, x_1, \ldots, x_N$.



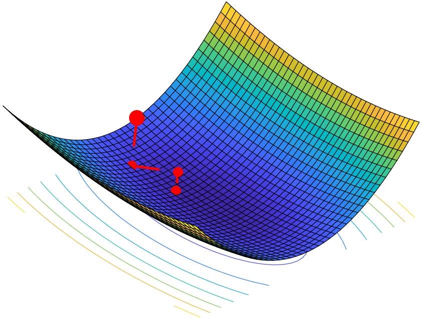
Gradient descent (stepsize α)

for $k = 0, 1, \ldots$ **do**

$$x_{k+1} = x_k - \alpha \nabla f(x_k)$$

end for

$$\min_{x\in\mathbb{R}^d} f(x) \qquad \rightarrow \text{ usually solved via } \textbf{iterative algorithm} \text{ generating sequence } x_0, x_1, \ldots, x_N.$$



Gradient descent (stepsize α)

for $k = 0, 1, \ldots$ **do**

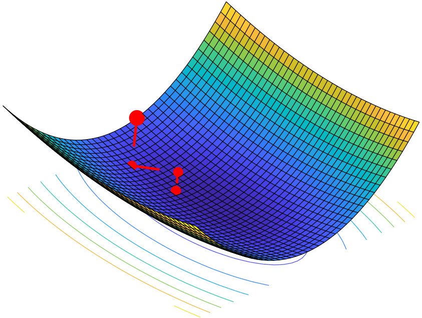
$$x_{k+1} = x_k - \alpha \nabla f(x_k)$$

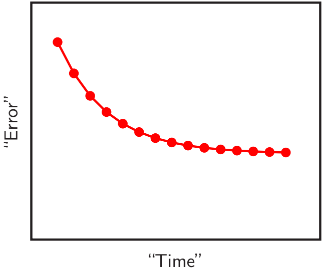
end for

What to expect from the output of the algorithm?

For instance: **bounds** on certain notions of "error": $f(x_k) - f(x_\star)$, $\|x_k - x_\star\|$, $\|\nabla f(x_k)\|$, etc.

$$\min_{x\in\mathbb{R}^d} f(x) \qquad \rightarrow \text{ usually solved via } \textbf{iterative algorithm} \text{ generating sequence } x_0, x_1, \ldots, x_N.$$

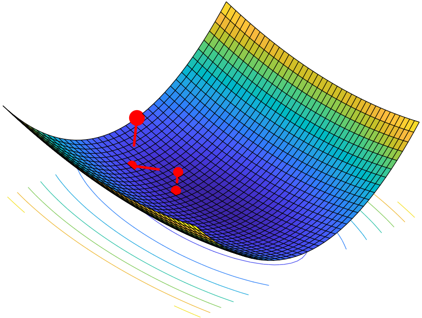


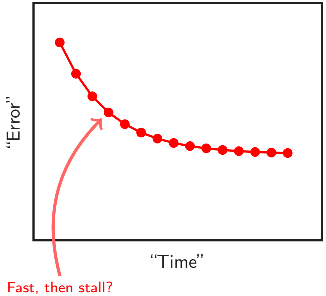


What to expect from the output of the algorithm?

For instance: **bounds** on certain notions of “error”: $f(x_k) - f(x_\star)$, $\|x_k - x_\star\|$, $\|\nabla f(x_k)\|$, etc.

$$\min_{x\in\mathbb{R}^d} f(x) \qquad \rightarrow \text{ usually solved via } \textbf{iterative algorithm} \text{ generating sequence } x_0, x_1, \ldots, x_N.$$

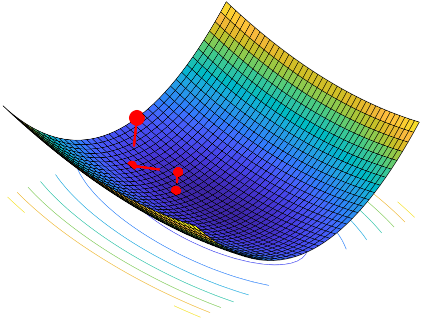


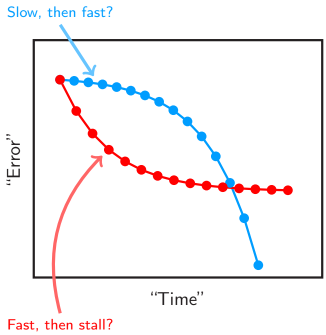


What to expect from the output of the algorithm?

For instance: **bounds** on certain notions of "error": $f(x_k) - f(x_\star)$, $\|x_k - x_\star\|$, $\|\nabla f(x_k)\|$, etc.

$$\min_{x \in \mathbb{R}^d} f(x) \qquad \rightarrow \text{ usually solved via } \textbf{iterative algorithm} \text{ generating sequence } x_0, x_1, \ldots, x_N.$$

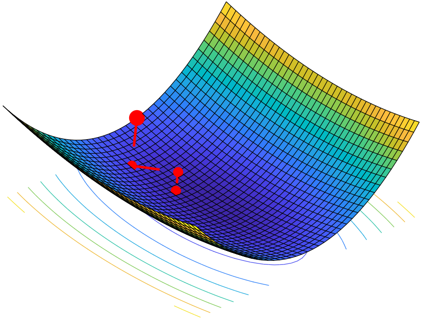


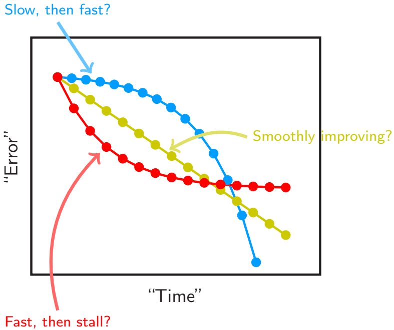


What to expect from the output of the algorithm?

For instance: **bounds** on certain notions of "error": $f(x_k) - f(x_\star)$, $\|x_k - x_\star\|$, $\|\nabla f(x_k)\|$, etc.

$$\min_{x\in\mathbb{R}^d} f(x) \qquad \to \text{ usually solved via } \textbf{iterative algorithm} \text{ generating sequence } x_0, x_1, \ldots, x_N.$$

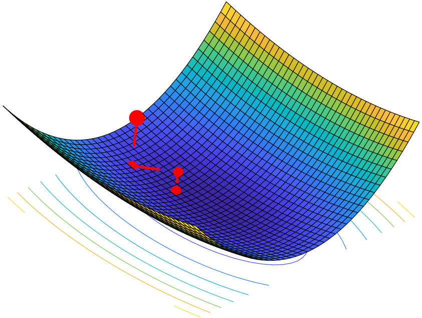


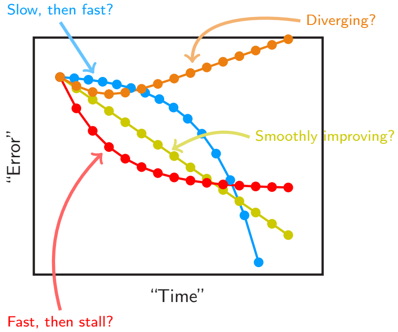


What to expect from the output of the algorithm?

For instance: **bounds** on certain notions of "error": $f(x_k) - f(x_\star)$, $\|x_k - x_\star\|$, $\|\nabla f(x_k)\|$, etc.

$\min_{x \in \mathbb{R}^d} f(x) \qquad \to$ usually solved via **iterative algorithm** generating sequence $x_0, x_1, \ldots, x_N$.





What to expect from the output of the algorithm?

For instance: **bounds** on certain notions of "error": $f(x_k) - f(x_\star)$, $\|x_k - x_\star\|$, $\|\nabla f(x_k)\|$, etc.

How to show that an algorithm works?

How to show that an algorithm works?

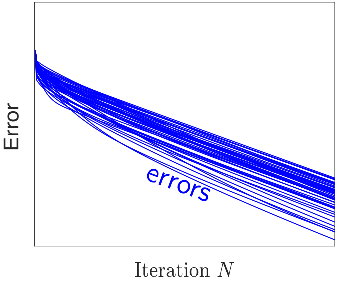
- ◇ Assumptions (no free lunch).

How to show that an algorithm works?

- ◇ Assumptions (no free lunch).
- ◇ Here: worst-case perspective.

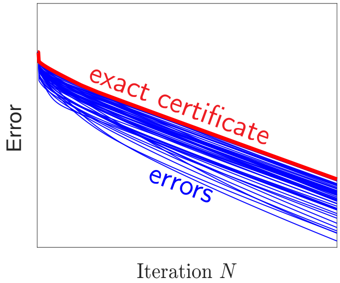
How to show that an algorithm works?

- Assumptions (no free lunch).
- Here: worst-case perspective.



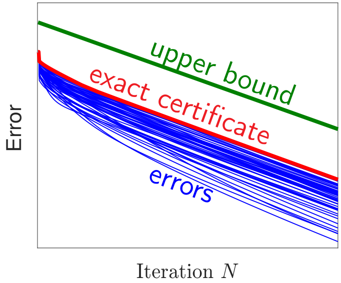
How to show that an algorithm works?

- Assumptions (no free lunch).
- Here: worst-case perspective.



How to show that an algorithm works?

- ◇ Assumptions (no free lunch).
- ◇ Here: worst-case perspective.



Organization and learning outcomes (1/3)

What we would like you to learn with this tutorial:

- ◇ Base understanding of performance estimation problems (PEPs),
- ◇ leverage PEPs for numerical comparison of base first-order algorithms,
- ◇ leverage PEPs to obtain simple convergence proofs (numerics and symbolics),
- ◇ leverage PEPs for algorithm design (numerics and symbolics).

Tutorial material:
`https://github.com/PerformanceEstimation/Tutorial-SIAM-OP26`

Organization and learning outcomes (2/3)

Part 1 (now, Wed June 3, 9:15 AM - 10:45 AM):

- ◇ Talk: understand the basics of PEPs: context, problem statement, SDPs, interpolation

  about 40mins
- ◇ Hands-on: numerical comparisons of FO methods using PEPs

  about 30mins
- ◇ Talk: dual formulations and obtaining performance bounds with proofs

  about 20mins

Organization and learning outcomes (3/3)

Part 2 (this PM, Wed June 3, 3:15 PM - 4:45 PM):

- Talk: proof structures

  about 15mins
- Hands-on: numerical and symbolic tools for proving convergence rates

  about 30mins
- Talk: designing first-order methods: principles and an example

  about 10mins
- Hands-on: numerical and symbolic tools for designing a first-order method

  about 20mins
- Discussions on going further

  about 15mins

Ressources

`https://github.com/PerformanceEstimation/Tutorial-SIAM-OP26`

Tools:

- Jupyter notebooks, or colab (here).
- Optimization tools/packages:
  - convex optimization interfacing/modeling tool: CVXPY.
  - Semidefinite (SDP) solvers: get academic MOSEK if possible (here).
- Symbolic computations:
  - SymPy [**OK for this minitutorial**], or consider Wolfram Engine (here+jupyter).
  - Mathematica is better (but expensive) — free (limited) version via cloud: here.

Please: do interrupt with questions!

Wooclap time!
(don't close it, you will need it later too!)

https://app.wooclap.com/ZHBGHP?from=instruction-slide

Software/material built with many great colleagues, among which:

Baptiste Goujaud

Céline Moucer

Julien Hendrickx

Aymeric Dieuleveut

Shuvomoy Das Gupta

Daniel Berg Thomsen